

Mastering JavaScript Functions

Declarations, Expressions, Parameters, Return Values & Default
Arguments

INSTRUCTOR

Web Development Series

DURATION

30 Minutes

What We're Covering Today

The fundamental building blocks of modular JavaScript code

1 Why We Need Functions & Core Benefits

2 Function Declarations & Execution

3 Parameters vs Arguments (Term Differences)

4 Return Statement vs `console.log`

5 Default Parameters & Function Expressions

6 Declarations vs Expressions (Hoisting)

What is a Function & Why Use It?

A named, reusable block of code that performs a specific task



Real-World Analogy

Think of a **Tea Machine**:

You press "Make Tea" (call function). The machine handles boiling water, mixing milk, and pouring tea automatically. You don't repeat the steps manually each time.

Reusability

Write logic once, execute anywhere as needed.

DRY Principle

"Don't Repeat Yourself" — eliminates duplicate code.

Easier Debugging

Fix bugs in one central location instead of many.

Function Declaration & Calling

Defining code vs. executing code

```
// 1. Declaring the function
function greet() {
  console.log("Hello, welcome to the
course!");
}

// 2. Calling (Executing) the function
greet(); // Runs code
greet(); // Runs again!
greet(); // Runs again!
```

⚠ Critical Distinction

Declaring a function **does nothing** by itself. You MUST call it using parentheses () to execute the block.

- **function keyword:** Tells JS we are creating a function.
- **Name:** Followed by identifier e.g. greet.
- **Body {}:** Contains all instructions to run on call.

Parameters vs. Arguments

Making functions dynamic with custom inputs

```
function greetUser(name) { // 'name' is a PARAMETER (Placeholder)
  console.log("Hello, " + name + "!");
}
```

```
greetUser("Ali"); // "Ali" is an ARGUMENT (Actual Value)
greetUser("Sara"); // "Sara" is an ARGUMENT
```

Parameter (Declaration)

Variable defined in function declaration acting as a placeholder.

Argument (Call)

Actual value passed into function when calling it.

Missing vs. Extra Arguments

How JavaScript handles argument count mismatches

Missing Arguments → undefined

```
function add(a, b) {  
  console.log(a + b);  
}  
  
add(5); // NaN  
// b becomes 'undefined'  
// 5 + undefined = NaN
```

Extra Arguments → Ignored

```
function add(a, b) {  
  console.log(a + b);  
}  
  
add(5, 10, 15); // 15  
// Extra argument (15)  
// is simply ignored
```

CRITICAL DISTINCTION

console.log vs. return

Printing output to console vs. passing data back to program

```
function addWithLog(a, b) { console.log(a + b); } // Prints, returns nothing
function addWithReturn(a, b) { return a + b; }    // Returns value back

let result1 = addWithLog(5, 10);    // Prints 15 | result1 = undefined
let result2 = addWithReturn(5, 10); // No print | result2 = 15

console.log("result1:", result1); // Output: undefined
console.log("result2:", result2); // Output: 15
```

console.log()

Only displays output in screen console. Function returns undefined.

return Statement

Passes data out of function so it can be stored in variables or reused.

Return Halts Execution Immediately

No code below a triggered return statement will execute

```
function checkAge(age) {  
  if (age ≥ 18) {  
    return "You are an adult.";  
  }  
  // Unreachable if age ≥ 18:  
  console.log("This line won't run for adults");  
  return "You are a minor.";  
}  
  
console.log(checkAge(20)); // "You are an adult."
```

Immediate Exit

The moment JavaScript hits return, it exits the function right away and hands back control to the caller.

Guard Clauses

This behavior is widely used to exit functions early if conditions aren't met.

Default Parameters

Providing fallback values for missing arguments

```
function calculateTotal(price, taxRate =  
0.05) {  
  return price + (price * taxRate);  
}  
  
// Uses default tax (5%):  
console.log(calculateTotal(100)); // 105  
  
// Overrides default (10%):  
console.log(calculateTotal(100, 0.10)); //  
110
```

✓ Why Use Default Params?

Prevents unwanted undefined or NaN bugs when callers omit optional arguments.

Default values only activate if the passed argument is **missing** or explicitly passed as undefined.

Function Expressions

Storing anonymous functions inside variables

```
// Anonymous Function Expression
const greet = function(name) {
  console.log("Hello, " + name + "!");
};

greet("Sara"); // Called via variable name

// Named Function Expression (debugging)
const multiply = function calc(a, b) {
  return a * b;
};
```

Anonymous Function

Function without its own name; assigned directly to a variable.

Semicolon Requirement

Ends with a semicolon ; because it is a standard variable assignment statement.

Declaration vs. Expression

The critical hoisting difference between both approaches

Feature	Function Declaration	Function Expression
Hoisting	Fully Hoisted (Call before defining)	Not Hoisted (Must define first)
Syntax	<code>function name() {}</code>	<code>const name = function() {}</code>
Named Required?	Yes	No (Can be anonymous)

```
// WORKS – Declaration is hoisted
sayHello();
function sayHello() { console.log("Hello!"); }
```

```
// FAILS – Expression not hoisted
sayHi(); // ❌ ReferenceError!
const sayHi = function() { console.log("Hi!"); };
```

Bonus Preview: Arrow Functions

Modern, concise function syntax introduced in ES6

```
// Modern ES6 Arrow Function
```

```
const add = (a, b) => {  
  return a + b;  
};
```

```
console.log(add(5, 10)); // 15
```

```
// Shorter single-line version:
```

```
const multiply = (a, b) => a * b;
```

Cleaner Syntax

Uses the "fat arrow" => instead of the function keyword.

We will cover arrow functions and their special this binding behavior in our dedicated ES6+ module!

Classroom Coding Task

Complete these 5 challenges and share solutions in the comments

- 1 Write function `isEven(num)` that returns `true` if even, `false` if odd.
- 2 Write function `calculateArea(length, width)` returning calculated area.
- 3 Add default parameter `width = 1` and call it with a single argument.
- 4 Convert `isEven` from a Function Declaration into a Function Expression.
- 5 **Bonus:** Write `greetTime(name, time="day")` logging `"Good [time], [name]!"`.

SUMMARY

Session Recap & Homework

Key takeaways to remember from Day 5

☰ Key Concepts Learned

- **Functions:** Reusable code blocks promoting DRY code.
- **Params vs Args:** Placeholders vs real input values.
- **Return:** Sends data out; halts function execution.
- **Default Params:** Provides safe fallback values.
- **Hoisting:** Declarations hoist; expressions do not.

✍ Homework Assignment

1. **Practice Task:** Complete all 5 coding tasks on Slide 13.
2. **Build 3 Custom Functions:** Write 3 functions for a real web project (e.g. discount calculator, form validator).
3. **Community Share:** Post your solutions in the comment section for feedback!

JavaScript Arrays & Methods

Basics, Accessing Elements, and essential methods: `push`, `pop`, `slice`, `splice`

Thank You for Watching!

Don't forget to Like, Share, & Subscribe for the full course series.

